



Static Analysis Vs Dynamic Analysis Tools

Both static analysers and dynamic analysers are widely used to find and fix bugs in programs. In this column, the author compares these analysers, and lists their advantages and disadvantages.

Both open source and enterprise software projects depend heavily on testing to find bugs. However, testing is costly in terms of the time and resources required. For this reason, using automatic analysers to find bugs has become a popular practice.

The types of tools

Static analysis tools check a program for potential bugs without actually executing the programs. For most programmers, the *lint* program immediately comes to mind when static analysers are mentioned. However, static analysis has come a long way from the good old days of the 1970s, when *lint* emerged. Though *lint*-like programs are widely used even today, static analysis has become very sophisticated, with very expensive tools available in the market today that can detect hard-to-find bugs with ease.

Static analysis tools can typically be used at any phase in the software development life-cycle. However, they are typically used once some source code is available for analysis. For example, there are static analysers that can analyse (structured or formalised) requirements specifications, UML diagrams, etc. So let's look at the advantages of static analysers:

- They find possible bugs in the code without actually executing the program—so you don't have to wait to get an executable program to find bugs.
- When you apply static analysers on the code early in the software development life-cycle, and find and fix bugs—that's good, because the earlier you find and fix bugs, the less money and effort it costs.
- Integrating and running static analysers is easy, and most tools are quick in finding problems and reporting them.

Dynamic analysers, on the other hand, check the program for bugs by actually executing the programs. So, dynamic analysis tools are used once executable code is available. These tools are extensively used by developers for the following reasons:

- Dynamic analysers find real issues (no false warnings).
- They find defects that actually result in software failure/problems.

- They aid in program debugging, tuning, tracing, etc — all of which are essential for the development and maintenance of complex software.

Desirable characteristics

Before we go ahead and compare these two kinds of tools, let's first get familiar with some terminology used in program analysis. In terms of finding errors in programs, there are four possibilities:

- False positives: These are false warnings or alarms that are raised when there is no real error in the code.
- True positives: This is an alarm that is raised for a real error in the code.
- False negatives: This is a missed error—i.e., no alarm was raised when there was a real error in the code.
- True negative: This is when no alarm is raised if there is no real error in the code.

If a tool warns of a real error in the code (a true positive), it is good, because that's why we're running the tool. If the code does not have a bug, and the tool does not report any warning (a true negative), that's also good. However, if the tool generates a warning when there is no error in the code (a false positive), it's annoying, and wastes developers' time, because they have to spend hours figuring out that the violation was not actually a problem in the code. Also, if the tool does not give any warning when there is an error in the code (a false negative), then that's also a problem since we want to find real bugs in code and that's why the tool is being run. If the tool misses most of the bugs, then it's not much use applying the tool. So, both false positives and false negatives are undesirable for program analysers.

Static analysers

The main problem with these is that they generate lots of warnings, and most of them are false positives (wrong warnings)! For example, if you run tools such as PC-Lint (C/C++), PMD (Java), or FxCop (C#) on a software with a million lines of code, you're likely to get close to a million violations!